Debugging

chapter 5 : Quit thinking and Look

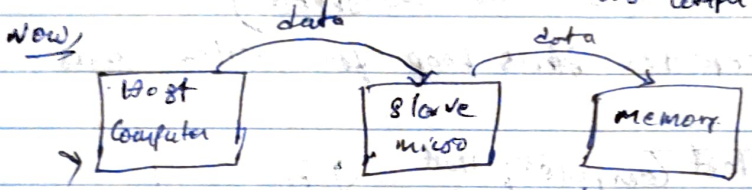
① Overview

Company "X" built a circuit board that plugged into personal computer and included its own slave microprocessor, with its own memory.



slave's program memory.

→ This could download data before host computer has started.



Here; slave would check for errors in the contents of memory)

If: the memory okay (✓), slave would start running,

else; slave would compile to host (i.e sent back to host)

print (program was corrupt; would not try to run).

Host Computer
↓
slave micro
↓
Memory
If : memory okay
slave starts running
Else: sent back to Host
print (Program was corrupt)

⇒ ② problem ; (ie draw back in this process).

- (Occasionally slave would report an error and stop after the download.) → this is intermittent ; happening one in ten system and only in some of the systems.
- This might occur even if the one bad byte out of 65,000 downloaded

this 'x' company assigned Junior hardware guys to fix this.

→ they Broke down these chain (Host computer ↔ Host Bus ↔ Slave micro ↔ memory Bus ↔ memory)

→ 1st they look into Host Comput ↔ Host Bus ↔ Slave micro

wrote a programe :

- wrote a value into one of the slave micros registers
- into one of the slave micro's register ← value is assigned
- ↓ first read that register back
- Result = ? (same as assigned)
- they repeated this Loop for millions times.

ex :- Host Computer ↔ Host Bus ↔ Slave micro

say :- 1) you have typed CHMES137 in your PC

↳ it is stored in one of slave micro's register

2) Read stored data : (it checks & backs the data that was registered)

3) If Read value = typed value :

same, test passed ... :

Else :

different, test failed, report error.

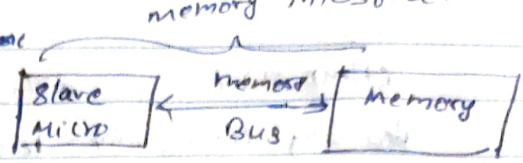
they tested this million times and they got the positive results

So, Host Comput ↔ Host Bus ↔ slave micro

~~Registers~~ no

this path has no errors.

→ Now, they looked into memory interface.



memory interface (i.e circuit diagram & the memory chip's datasheet).

Actually, we get the error: If the value stored in the memory is not same as the typed value and the result

{ typed CHMES137
↓ Host Bus
slave micro
↓ memory Bus
memory

{ till here no errors }

{ Now, if the value stored in the memory as "CHMES137" this can store for some time

If we ask the value in that period of time we get some input value

If we ask after End time of "memory storage time" we get error }

memory

For value in time = 5 sec:

Print (input value)

Else:

print error

- what is actually happening;

memory Chip has address wires and a control wire.

i) when input value is assigned (this I/P value is also kept on some location on address wires by micro)

ii) When you ask result value:

memory looks at the adress wires, and location micro and grabs what memory says.

Here; there Adress wire can store for default hold time of then that memory looks to the net location.

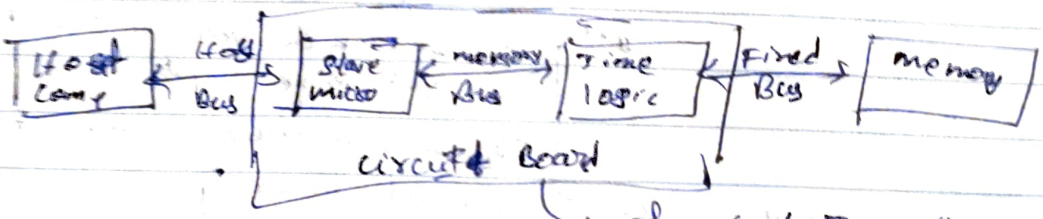
Adress wires: [1, 2, 3, ... 99, 100 ...]

Control wires:

| setup time | edge | hold time |

Let;
CHMOS137 on address wire 99 and can hold 1 nsec
Set during this 1 nsec ; memory point same address wire and gives signal to micro of CHMOS137.
But; Senior Engineers pointed out that in some cases even after 3 nsec the value is changing (i.e memory is point out the next address wire).

→ Solution
they decided to fix the timing ;
prepared "hand-wired prototype board" (in Circuit board
↓
H/w micro & memory



this took 7 months to design and built.

Even after using circuit board ; occasionally memory used to give errors.

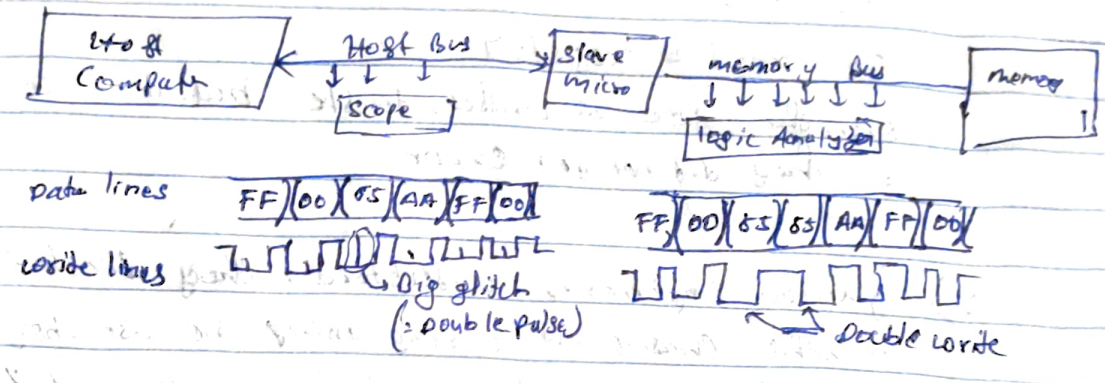
→ One Senior Engineer got into this job

- He took out the heavy duty logic analyzer.
He wrote a test that put a known, regular pattern of data in a repeating loop of "00 55 AA FF"
- He expected to get some error like "00 54 AA FF"
but system output was "00 55 55 AA FF"

He understood ("It was not wrong data was being written in, but right data was being written in twice").

He went back to the host bus, and put a scope on a few signals, He found out there was noise on the write line

Reason; Because of other circuitry on the board, it had small glitch in the middle. that was occasionally big enough to make the pulse look like two pulses



Conclusion

when you write a register and read it back (which is done by junior engineers), using two write pulses just writes it twice; it still reads back correctly.

But when downloading data through the chip, where each write sends another byte of data to the next location in memory.

using two write pulses;
the second write goes to the next location and every thing after that is off by one address.

ie.

SIP

Address	0	1	2	3
data	00	55	AA	FF

what happened; (memory holds)

Address	0	1	2	3	4
Data	00	55	55	AA	FF
Because	Byte 00 one pulse	Byte 55 two pulse		Byte AA one pulse	Byte FF one pulse

* the extra pulse stored it a second time, that pushed the address forward by one.

Error of Junior Engineers

They just asked the register and read.

If read = register

Even the pulse as written double, but it is correct what as read so, they did not get error.

2) Junior Engineers understood, they wasted several months Chasing the wrong thing because they ~~guessed~~ guessed at the failure instead of looking at it.

(II.) See the failure

→ As I said in take Away ; Look at the failure, buy asking in number questions causing the failure before trying to fix failure.

→ In the above eg :
Junior Engineer never looked ~~into~~ at the memory being written incorrectly as a result of bad timing.

(III) See the Details

→ Each time you look into the system to see the failure, you learn more about what's failing.

→ this will help you decide where to look even deeper to get more detail.

→ By this process, you get enough detail that it makes sense to look at the design and figure out what the cause of the problem is

eg: well pump. (compressor) in garage

they that the noise is due to the pump and they fixed that by spending lot of money, but eventually ended up the noise again.

if they have went to basement to listen the source of noise they might have understood the cause of the problem.

(IV) Now you see it, Now you Don't

→ After, seeing the details cause of failure at detail (i.e low level), and you fix the bug and results are positive (i.e you actually fixed the right bug). You can see errors doesn't happen anymore.

→ In previous eg; when senior engineer fixed the noise problem on slave microprocessor, the glitch in the write pulse was gone.

Take away

" Never guess, the error is because of something might have failed".

Look the failure at very-deep and confirm the cause of failure before fixing it. ✓

" We are Chemical Engineers, Everything we work, costs million dollars so, before fixing the errors try to analyse the error at lower level before going to Corrective Action ".